

Modern Operating Systems — Chapter 2 Processes and Threads

Exercises and Answers

Andrew S. Tanenbaum and Herbert Bos

Contents

Chapter 2: Processes and Threads	2
Exercise 1	2
Exercise 2	2
Exercise 3	3
Exercise 4	3
Exercise 5	3
Exercise 6	4
Exercise 7	4
Exercise 8	5
Exercise 9	5
Exercise 10	5
Exercise 11	6
Exercise 12	6
Exercise 13	7
Exercise 14	7
Exercise 15	7

Exercise 16	8
Exercise 17	8
Exercise 18	9
Exercise 19	9
Exercise 20	9
Exercise 21	10
Exercise 22	10
Exercise 23	11
Exercise 24	11
Exercise 25	11
Exercise 26	12
Exercise 27	12
Exercise 28	12
Exercise 29	13
Exercise 30	13
Exercise 31	14
Exercise 32	14
Exercise 33	15
Exercise 34	15
Exercise 35	15
Exercise 36	16

Exercise 37	16
Exercise 38	17
Exercise 39	17
Exercise 40	17
Exercise 41	18
Exercise 42	18
Exercise 43	19
Exercise 44	19
Exercise 45	20
Exercise 46	20
Exercise 47	21
Exercise 48	21
Exercise 49	21
Exercise 50	22
Exercise 51	22
Exercise 52	22
Exercise 53	23
Exercise 54	23
Exercise 55	23
Exercise 56	24
Exercise 57	24

Exercise 58	25
Exercise 59	26
Exercise 60	27
Exercise 61	29
Exercise 62	30
Exercise 63	31
Exercise 64	32
Exercise 65	34

Chapter 2: Processes and Threads

Source note. Questions were extracted from the Chapter 2 problem section of *Modern Operating Systems*, 4th edition. Answers 1–57 were matched to the Chapter 2 answer section of the provided `MOS_4th_answer.pdf`. Exercises 58–65 had no matching answer in that answer section, so their answers are clearly labeled as prepared technical answers rather than answer-manual text.

Exercise 1

Question

In Fig. 2-2, three process states are shown. In theory, with three states, there could be six transitions, two out of each state. However, only four transitions are shown. Are there any circumstances in which either or both of the missing transitions might occur?

Answer

The transition from blocked to running is conceivable. Suppose that a process is blocked on I/O and the I/O finishes. If the CPU is otherwise idle, the process could go directly from blocked to running. The other missing transition, from ready to blocked, is impossible. A ready process cannot do I/O or anything else that might block it. Only a running process can block.

Source: Answer manual.

Exercise 2

Question

Suppose that you were to design an advanced computer architecture that did process switching in hardware, instead of having interrupts. What information would the CPU need? Describe how the hardware process switching might work.

Answer

You could have a register containing a pointer to the current process-table entry. When I/O completed, the CPU would store the current machine state in the current process-table entry. Then it would go to the interrupt vector for the interrupting device and fetch a pointer to another process-table entry (the service procedure). This process would then be started up.

Source: Answer manual.

Exercise 3

Question

On all current computers, at least part of the interrupt handlers are written in assembly language. Why?

Answer

Generally, high-level languages do not allow the kind of access to CPU hardware that is required. For instance, an interrupt handler may be required to enable and disable the interrupt servicing a particular device, or to manipulate data within a process' stack area. Also, interrupt service routines must execute as rapidly as possible.

Source: Answer manual.

Exercise 4

Question

When an interrupt or a system call transfers control to the operating system, a kernel stack area separate from the stack of the interrupted process is generally used. Why?

Answer

There are several reasons for using a separate stack for the kernel. Two of them are as follows. First, you do not want the operating system to crash because a poorly written user program does not allow for enough stack space. Second, if the kernel leaves stack data in a user program's memory space upon return from a system call, a malicious user might be able to use this data to find out information about other processes.

Source: Answer manual.

Exercise 5

Question

A computer system has enough room to hold five programs in its main memory. These programs are idle waiting for I/O half the time. What fraction of the CPU time is wasted?

Answer

The chance that all five processes are idle is $1/32$, so the CPU idle time is $1/32$.

Source: Answer manual.

Exercise 6

Question

A computer has 4 GB of RAM of which the operating system occupies 512 MB. The processes are all 256 MB (for simplicity) and have the same characteristics. If the goal is 99% CPU utilization, what is the maximum I/O wait that can be tolerated?

Answer

There is enough room for 14 processes in memory. If a process has an I/O of p , then the probability that they are all waiting for I/O is p^{14} . By equating this to 0.01, we get the equation $p^{14} = 0.01$. Solving this, we get $p = 0.72$, so we can tolerate processes with up to 72% I/O wait.

Source: Answer manual.

Exercise 7

Question

Multiple jobs can run in parallel and finish faster than if they had run sequentially. Suppose that two jobs, each needing 20 minutes of CPU time, start simultaneously. How long will the last one take to complete if they run sequentially? How long if they run in parallel? Assume 50% I/O wait.

Answer

If each job has 50% I/O wait, then it will take 40 minutes to complete in the absence of competition. If run sequentially, the second one will finish 80 minutes after the first one starts. With two jobs, the approximate CPU utilization is $1 - 0.5^2$. Thus, each one gets 0.375 CPU minute per minute of real time. To accumulate 20 minutes of CPU time, a job must run for $20/0.375$ minutes, or about 53.33 minutes. Thus running sequentially the jobs finish after 80 minutes, but running in parallel they finish after 53.33 minutes.

Source: Answer manual.

Exercise 8

Question

Consider a multiprogrammed system with degree of 6 (i.e., six programs in memory at the same time). Assume that each process spends 40% of its time waiting for I/O. What will be the CPU utilization?

Answer

The probability that all processes are waiting for I/O is 0.4^6 which is 0.004096. Therefore, CPU utilization = $1 - 0.004096 = 0.995904$.

Source: Answer manual.

Exercise 9

Question

Assume that you are trying to download a large 2-GB file from the Internet. The file is available from a set of mirror servers, each of which can deliver a subset of the file's bytes; assume that a given request specifies the starting and ending bytes of the file. Explain how you might use threads to improve the download time.

Answer

The client process can create separate threads; each thread can fetch a different part of the file from one of the mirror servers. This can help reduce downtime. Of course, there is a single network link being shared by all threads. This link can become a bottleneck as the number of threads becomes very large.

Source: Answer manual.

Exercise 10

Question

In the text it was stated that the model of Fig. 2-11(a) was not suited to a file server using a cache in memory. Why not? Could each process have its own cache?

Answer

It would be difficult, if not impossible, to keep the file system consistent. Suppose that a client process sends a request to server process 1 to update a file. This process updates the

cache entry in its memory. Shortly thereafter, another client process sends a request to server 2 to read that file. Unfortunately, if the file is also cached there, server 2, in its innocence, will return obsolete data. If the first process writes the file through to the disk after caching it, and server 2 checks the disk on every read to see if its cached copy is up-to-date, the system can be made to work, but it is precisely all these disk accesses that the caching system is trying to avoid.

Source: Answer manual.

Exercise 11

Question

If a multithreaded process forks, a problem occurs if the child gets copies of all the parent's threads. Suppose that one of the original threads was waiting for keyboard input. Now two threads are waiting for keyboard input, one in each process. Does this problem ever occur in single-threaded processes?

Answer

No. If a single-threaded process is blocked on the keyboard, it cannot fork.

Source: Answer manual.

Exercise 12

Question

In Fig. 2-8, a multithreaded Web server is shown. If the only way to read from a file is the normal blocking read system call, do you think user-level threads or kernel-level threads are being used for the Web server? Why?

Answer

A worker thread will block when it has to read a Web page from the disk. If user-level threads are being used, this action will block the entire process, destroying the value of multithreading. Thus it is essential that kernel threads are used to permit some threads to block without affecting the others.

Source: Answer manual.

Exercise 13

Question

In the text, we described a multithreaded Web server, showing why it is better than a single-threaded server and a finite-state machine server. Are there any circumstances in which a single-threaded server might be better? Give an example.

Answer

Yes. If the server is entirely CPU bound, there is no need to have multiple threads. It just adds unnecessary complexity. As an example, consider a telephone directory assistance number (like 555-1212) for an area with 1 million people. If each (name, telephone number) record is, say, 64 characters, the entire database takes 64 megabytes and can easily be kept in the server's memory to provide fast lookup.

Source: Answer manual.

Exercise 14

Question

In Fig. 2-12 the register set is listed as a per-thread rather than a per-process item. Why? After all, the machine has only one set of registers.

Answer

When a thread is stopped, it has values in the registers. They must be saved, just as when the process is stopped. the registers must be saved. Multiprogramming threads is no different than multiprogramming processes, so each thread needs its own register save area.

Source: Answer manual.

Exercise 15

Question

Why would a thread ever voluntarily give up the CPU by calling `thread yield`? After all, since there is no periodic clock interrupt, it may never get the CPU back.

Answer

Threads in a process cooperate. They are not hostile to one another. If yielding is needed for the good of the application, then a thread will yield. After all, it is usually the same

programmer who writes the code for all of them.

Source: Answer manual.

Exercise 16

Question

Can a thread ever be preempted by a clock interrupt? If so, under what circumstances? If not, why not?

Answer

User-level threads cannot be preempted by the clock unless the whole process' quantum has been used up (although transparent clock interrupts can happen). Kernel-level threads can be preempted individually. In the latter case, if a thread runs too long, the clock will interrupt the current process and thus the current thread. The kernel is free to pick a different thread from the same process to run next if it so desires.

Source: Answer manual.

Exercise 17

Question

In this problem you are to compare reading a file using a single-threaded file server and a multithreaded server. It takes 12 msec to get a request for work, dispatch it, and do the rest of the necessary processing, assuming that the data needed are in the block cache. If a disk operation is needed, as is the case one-third of the time, an additional 75 msec is required, during which time the thread sleeps. How many requests/sec can the server handle if it is single threaded? If it is multithreaded?

Answer

In the single-threaded case, the cache hits take 12 msec and cache misses take 87 msec. The weighted average is $\frac{2}{3} \times 12 + \frac{1}{3} \times 87$. Thus, the mean request takes 37 msec and the server can do about 27 per second. For a multithreaded server, all the waiting for the disk is overlapped, so every request takes 12 msec, and the server can handle $83 \frac{1}{3}$ requests per second.

Source: Answer manual.

Exercise 18

Question

What is the biggest advantage of implementing threads in user space? What is the biggest disadvantage?

Answer

The biggest advantage is the efficiency. No traps to the kernel are needed to switch threads. The biggest disadvantage is that if one thread blocks, the entire process blocks.

Source: Answer manual.

Exercise 19

Question

In Fig. 2-15 the thread creations and messages printed by the threads are interleaved at random. Is there a way to force the order to be strictly thread 1 created, thread 1 prints message, thread 1 exits, thread 2 created, thread 2 prints message, thread 2 exists, and so on? If so, how? If not, why not?

Answer

Yes, it can be done. After each call to pthread create, the main program could do a pthread join to wait until the thread just created has exited before creating the next thread.

Source: Answer manual.

Exercise 20

Question

In the discussion on global variables in threads, we used a procedure create global to allocate storage for a pointer to the variable, rather than the variable itself. Is this essential, or could the procedures work with the values themselves just as well?

Answer

The pointers are really necessary because the size of the global variable is unknown. It could be anything from a character to an array of floating-point numbers. If the value were stored, one would have to give the size to create global, which is all right, but what type should the second parameter of set global be, and what type should the value of read global

be?

Source: Answer manual.

Exercise 21

Question

Consider a system in which threads are implemented entirely in user space, with the run-time system getting a clock interrupt once a second. Suppose that a clock interrupt occurs while some thread is executing in the run-time system. What problem might occur? Can you suggest a way to solve it?

Answer

It could happen that the runtime system is precisely at the point of blocking or unblocking a thread, and is busy manipulating the scheduling queues. This would be a very inopportune moment for the clock interrupt handler to begin inspecting those queues to see if it was time to do thread switching, since they might be in an inconsistent state. One solution is to set a flag when the runtime system is entered. The clock handler would see this and set its own flag, then return. When the runtime system finished, it would check the clock flag, see that a clock interrupt occurred, and now run the clock handler.

Source: Answer manual.

Exercise 22

Question

Suppose that an operating system does not have anything like the select system call to see in advance if it is safe to read from a file, pipe, or device, but it does allow alarm clocks to be set that interrupt blocked system calls. Is it possible to implement a threads package in user space under these conditions? Discuss.

Answer

Yes it is possible, but inefficient. A thread wanting to do a system call first sets an alarm timer, then does the call. If the call blocks, the timer returns control to the threads package. Of course, most of the time the call will not block, and the timer has to be cleared. Thus each system call that might block has to be executed as three system calls. If timers go off prematurely, all kinds of problems develop. This is not an attractive way to build a threads package.

Source: Answer manual.

Exercise 23

Question

Does the busy waiting solution using the turn variable (Fig. 2-23) work when the two processes are running on a shared-memory multiprocessor, that is, two CPUs sharing a common memory?

Answer

Yes, it still works, but it still is busy waiting, of course.

Source: Answer manual.

Exercise 24

Question

Does Peterson's solution to the mutual-exclusion problem shown in Fig. 2-24 work when process scheduling is preemptive? How about when it is nonpreemptive?

Answer

It certainly works with preemptive scheduling. In fact, it was designed for that case. When scheduling is nonpreemptive, it might fail. Consider the case in which turn is initially 0 but process 1 runs first. It will just loop forever and never release the CPU.

Source: Answer manual.

Exercise 25

Question

Can the priority inversion problem discussed in Sec. 2.3.4 happen with user-level threads? Why or why not?

Answer

The priority inversion problem occurs when a low-priority process is in its critical region and suddenly a high-priority process becomes ready and is scheduled. If it uses busy waiting, it will run forever. With user-level threads, it cannot happen that a low-priority thread is suddenly preempted to allow a high-priority thread run. There is no preemption. With kernel-level threads this problem can arise.

Source: Answer manual.

Exercise 26

Question

In Sec. 2.3.4, a situation with a high-priority process, H, and a low-priority process, L, was described, which led to H looping forever. Does the same problem occur if roundrobin scheduling is used instead of priority scheduling? Discuss.

Answer

With round-robin scheduling it works. Sooner or later L will run, and eventually it will leave its critical region. The point is, with priority scheduling, L never gets to run at all; with round robin, it gets a normal time slice periodically, so it has the chance to leave its critical region.

Source: Answer manual.

Exercise 27

Question

In a system with threads, is there one stack per thread or one stack per process when user-level threads are used? What about when kernel-level threads are used? Explain.

Answer

Each thread calls procedures on its own, so it must have its own stack for the local variables, return addresses, and so on. This is equally true for user-level threads as for kernel-level threads.

Source: Answer manual.

Exercise 28

Question

When a computer is being developed, it is usually first simulated by a program that runs one instruction at a time. Even multiprocessors are simulated strictly sequentially like this. Is it possible for a race condition to occur when there are no simultaneous events like this?

Answer

Yes. The simulated computer could be multiprogrammed. For example, while process A is running, it reads out some shared variable. Then a simulated clock tick happens and

process B runs. It also reads out the same variable. Then it adds 1 to the variable. When process A runs, if it also adds 1 to the variable, we have a race condition.

Source: Answer manual.

Exercise 29

Question

The producer-consumer problem can be extended to a system with multiple producers and consumers that write (or read) to (from) one shared buffer. Assume that each producer and consumer runs in its own thread. Will the solution presented in Fig. 2-28, using semaphores, work for this system?

Answer

Yes, it will work as is. At a given time instant, only one producer (consumer) can add (remove) an item to (from) the buffer.

Source: Answer manual.

Exercise 30

Question

Consider the following solution to the mutual-exclusion problem involving two processes P0 and P1. Assume that the variable turn is initialized to 0. Process P0's code is presented below. `/* Other code */ while (turn != 0) { } /* Do nothing and wait. */ Critical Section /* . . . */ turn = 0; /* Other code */` For process P1, replace 0 by 1 in above code. Determine if the solution meets all the required conditions for a correct mutual-exclusion solution.

Answer

The solution satisfies mutual exclusion since it is not possible for both processes to be in their critical section. That is, when turn is 0, P0 can execute its critical section, but not P1. Likewise, when turn is 1. However, this assumes P0 must run first. If P1 produces something and it puts it in a buffer, then while P0 can get into its critical section, it will find the buffer empty and block. Also, this solution requires strict alternation of the two processes, which is undesirable.

Source: Answer manual.

Exercise 31

Question

How could an operating system that can disable interrupts implement semaphores?

Answer

To do a semaphore operation, the operating system first disables interrupts. Then it reads the value of the semaphore. If it is doing a down and the semaphore is equal to zero, it puts the calling process on a list of blocked processes associated with the semaphore. If it is doing an up, it must check to see if any processes are blocked on the semaphore. If one or more processes are blocked, one of them is removed from the list of blocked processes and made runnable. When all these operations have been completed, interrupts can be enabled again.

Source: Answer manual.

Exercise 32

Question

Show how counting semaphores (i.e., semaphores that can hold an arbitrary value) can be implemented using only binary semaphores and ordinary machine instructions.

Answer

Associated with each counting semaphore are two binary semaphores, M, used for mutual exclusion, and B, used for blocking. Also associated with each counting semaphore is a counter that holds the number of ups minus the number of downs, and a list of processes blocked on that semaphore. To implement down, a process first gains exclusive access to the semaphores, counter, and list by doing a down on M. It then decrements the counter. If it is zero or more, it just does an up on M and exits. If M is negative, the process is put on the list of blocked processes. Then an up is done on M and a down is done on B to block the process. To implement up, first M is downed to get mutual exclusion, and then the counter is incremented. If it is more than zero, no one was blocked, so all that needs to be done is to up M. If, however, the counter is now negative or zero, some process must be removed from the list. Finally, an up is done on B and M in that order.

Source: Answer manual.

Exercise 33

Question

If a system has only two processes, does it make sense to use a barrier to synchronize them? Why or why not?

Answer

If the program operates in phases and neither process may enter the next phase until both are finished with the current phase, it makes perfect sense to use a barrier.

Source: Answer manual.

Exercise 34

Question

Can two threads in the same process synchronize using a kernel semaphore if the threads are implemented by the kernel? What if they are implemented in user space? Assume that no threads in any other processes have access to the semaphore. Discuss your answers.

Answer

With kernel threads, a thread can block on a semaphore and the kernel can run some other thread in the same process. Consequently, there is no problem using semaphores. With user-level threads, when one thread blocks on a semaphore, the kernel thinks the entire process is blocked and does not run it ever again. Consequently, the process fails.

Source: Answer manual.

Exercise 35

Question

Synchronization within monitors uses condition variables and two special operations, wait and signal. A more general form of synchronization would be to have a single primitive, waituntil, that had an arbitrary Boolean predicate as parameter. Thus, one could say, for example, waituntil $x < 0$ or $y + z < n$. The signal primitive would no longer be needed. This scheme is clearly more general than that of Hoare or Brinch Hansen, but it is not used. Why not? (Hint: Think about the implementation.)

Answer

It is very expensive to implement. Each time any variable that appears in a predicate on which some process is waiting changes, the run-time system must re-evaluate the predicate to see if the process can be unblocked. With the Hoare and Brinch Hansen monitors, processes can only be awakened on a signal primitive.

Source: Answer manual.

Exercise 36

Question

A fast-food restaurant has four kinds of employees: (1) order takers, who take customers' orders; (2) cooks, who prepare the food; (3) packaging specialists, who stuff the food into bags; and (4) cashiers, who give the bags to customers and take their money. Each employee can be regarded as a communicating sequential process. What form of interprocess communication do they use? Relate this model to processes in UNIX.

Answer

The employees communicate by passing messages: orders, food, and bags in this case. In UNIX terms, the four processes are connected by pipes.

Source: Answer manual.

Exercise 37

Question

Suppose that we have a message-passing system using mailboxes. When sending to a full mailbox or trying to receive from an empty one, a process does not block. Instead, it gets an error code back. The process responds to the error code by just trying again, over and over, until it succeeds. Does this scheme lead to race conditions?

Answer

It does not lead to race conditions (nothing is ever lost), but it is effectively busy waiting.

Source: Answer manual.

Exercise 38

Question

The CDC 6600 computers could handle up to 10 I/O processes simultaneously using an interesting form of round-robin scheduling called processor sharing. A process switch occurred after each instruction, so instruction 1 came from process 1, instruction 2 came from process 2, etc. The process switching was done by special hardware, and the overhead was zero. If a process needed T sec to complete in the absence of competition, how much time would it need if processor sharing was used with n processes?

Answer

It will take nT sec.

Source: Answer manual.

Exercise 39

Question

Consider the following piece of C code: `void main( ) { fork( ); fork( ); exit( ); }` How many child processes are created upon execution of this program?

Answer

Three processes are created. After the initial process forks, there are two processes running, a parent and a child. Each of them then forks, creating two additional processes. Then all the processes exit.

Source: Answer manual.

Exercise 40

Question

Round-robin schedulers normally maintain a list of all runnable processes, with each process occurring exactly once in the list. What would happen if a process occurred twice in the list? Can you think of any reason for allowing this?

Answer

If a process occurs multiple times in the list, it will get multiple quanta per cycle. This approach could be used to give more important processes a larger share of the CPU. But

when the process blocks, all entries had better be removed from the list of runnable processes.

Source: Answer manual.

Exercise 41

Question

Can a measure of whether a process is likely to be CPU bound or I/O bound be determined by analyzing source code? How can this be determined at run time?

Answer

In simple cases it may be possible to see if I/O will be limiting by looking at source code. For instance a program that reads all its input files into buffers at the start will probably not be I/O bound, but a problem that reads and writes incrementally to a number of different files (such as a compiler) is likely to be I/O bound. If the operating system provides a facility such as the UNIX `ps` command that can tell you the amount of CPU time used by a program, you can compare this with the total time to complete execution of the program. This is, of course, most meaningful on a system where you are the only user.

Source: Answer manual.

Exercise 42

Question

Explain how time quantum value and context switching time affect each other, in a round-robin scheduling algorithm.

Answer

If the context switching time is large, then the time quantum value has to be proportionally large. Otherwise, the overhead of context switching can be quite high. Choosing large time quantum values can lead to an inefficient system if the typical CPU burst times are less than the time quantum. If context switching is very small or negligible, then the time quantum value can be chosen with more freedom.

Source: Answer manual.

Exercise 43

Question

Measurements of a certain system have shown that the average process runs for a time T before blocking on I/O. A process switch requires a time S , which is effectively wasted (overhead). For round-robin scheduling with quantum Q , give a formula for the CPU efficiency for each of the following:

- (a) $Q = \infty$
- (b) $Q > T$
- (c) $S < Q < T$
- (d) $Q = S$
- (e) Q nearly 0

Answer

The CPU efficiency is the useful CPU time divided by the total CPU time. When $Q \geq T$, the basic cycle is for the process to run for T and undergo a process switch for S . Thus, (a) and (b) have an efficiency of $T / (S + T)$. When the quantum is shorter than T , each run of T will require T / Q process switches, wasting a time ST / Q . The efficiency here is then $T / (T + ST / Q)$ which reduces to $Q / (Q + S)$, which is the answer to (c). For (d), we just substitute Q for S and find that the efficiency is 50%. Finally, for (e), as $Q \rightarrow 0$ the efficiency goes to 0.

Source: Answer manual.

Exercise 44

Question

Five jobs are waiting to be run. Their expected run times are 9, 6, 3, 5, and X . In what order should they be run to minimize average response time? (Your answer will depend on X .)

Answer

Shortest job first is the way to minimize average response time. $0 < X \leq 3$: $X, 3, 5, 6, 9$. $3 < X \leq 5$: $3, X, 5, 6, 9$. $5 < X \leq 6$: $3, 5, X, 6, 9$. $6 < X \leq 9$: $3, 5, 6, X, 9$. $X > 9$: $3, 5, 6, 9, X$.

Source: Answer manual.

Exercise 45

Question

Five batch jobs. A through E, arrive at a computer center at almost the same time. They have estimated running times of 10, 6, 2, 4, and 8 minutes. Their (externally determined) priorities are 3, 5, 2, 1, and 4, respectively, with 5 being the highest priority. For each of the following scheduling algorithms, determine the mean process turnaround time. Ignore process switching overhead.

- (a) Round robin.
- (b) Priority scheduling.
- (c) First-come, first-served (run in order 10, 6, 2, 4, 8).
- (d) Shortest job first. For (a), assume that the system is multiprogrammed, and that each job gets its fair share of the CPU. For (b) through (d), assume that only one job at a time runs, until it finishes. All jobs are completely CPU bound.

Answer

For round robin, during the first 10 minutes each job gets $1/5$ of the CPU. At the end of 10 minutes, C finishes. During the next 8 minutes, each job gets $1/4$ of the CPU, after which time D finishes. Then each of the three remaining jobs gets $1/3$ of the CPU for 6 minutes, until B finishes, and so on. The finishing times for the five jobs are 10, 18, 24, 28, and 30, for an average of 22 minutes. For priority scheduling, B is run first. After 6 minutes it is finished. The other jobs finish at 14, 24, 26, and 30, for an average of 18.8 minutes. If the jobs run in the order A through E, they finish at 10, 16, 18, 22, and 30, for an average of 19.2 minutes. Finally, shortest job first yields finishing times of 2, 6, 12, 20, and 30, for an average of 14 minutes.

Source: Answer manual.

Exercise 46

Question

A process running on CTSS needs 30 quanta to complete. How many times must it be swapped in, including the very first time (before it has run at all)?

Answer

The first time it gets 1 quantum. On succeeding runs it gets 2, 4, 8, and 15, so it must be swapped in 5 times.

Source: Answer manual.

Exercise 47

Question

Consider a real-time system with two voice calls of periodicity 5 msec each with CPU time per call of 1 msec, and one video stream of periodicity 33 ms with CPU time per call of 11 msec. Is this system schedulable?

Answer

Each voice call needs 200 samples of 1 msec or 200 msec. Together they use 400 msec of CPU time. The video needs 11 msec $33 \frac{1}{3}$ times a second for a total of about 367 msec. The sum is 767 msec per second of real time so the system is schedulable.

Source: Answer manual.

Exercise 48

Question

For the above problem, can another video stream be added and have the system still be schedulable?

Answer

Another video stream consumes 367 msec of time per second for a total of 1134 msec per second of real time so the system is not schedulable.

Source: Answer manual.

Exercise 49

Question

The aging algorithm with $a = 1/2$ is being used to predict run times. The previous four runs, from oldest to most recent, are 40, 20, 40, and 15 msec. What is the prediction of the next time?

Answer

The sequence of predictions is 40, 30, 35, and now 25.

Source: Answer manual.

Exercise 50

Question

A soft real-time system has four periodic events with periods of 50, 100, 200, and 250 msec each. Suppose that the four events require 35, 20, 10, and x msec of CPU time, respectively. What is the largest value of x for which the system is schedulable?

Answer

The fraction of the CPU used is $35/50 + 20/100 + 10/200 + x/250$. To be schedulable, this must be less than 1. Thus x must be less than 12.5 msec.

Source: Answer manual.

Exercise 51

Question

In the dining philosophers problem, let the following protocol be used: An even-numbered philosopher always picks up his left fork before picking up his right fork; an odd-numbered philosopher always picks up his right fork before picking up his left fork. Will this protocol guarantee deadlock-free operation?

Answer

Yes. There will be always at least one fork free and at least one philosopher that can obtain both forks simultaneously. Hence, there will be no deadlock. You can try this for $N = 2$, $N = 3$ and $N = 4$ and then generalize.

Source: Answer manual.

Exercise 52

Question

A real-time system needs to handle two voice calls that each run every 6 msec and consume 1 msec of CPU time per burst, plus one video at 25 frames/sec, with each frame requiring 20 msec of CPU time. Is this system schedulable?

Answer

Each voice call runs 166.67 times/second and uses up 1 msec per burst, so each voice call needs 166.67 msec per second or 333.33 msec for the two of them. The video runs 25 times

a second and uses up 20 msec each time, for a total of 500 msec per second. Together they consume 833.33 msec per second, so there is time left over and the system is schedulable.

Source: Answer manual.

Exercise 53

Question

Consider a system in which it is desired to separate policy and mechanism for the scheduling of kernel threads. Propose a means of achieving this goal.

Answer

The kernel could schedule processes by any means it wishes, but within each process it runs threads strictly in priority order. By letting the user process set the priority of its own threads, the user controls the policy but the kernel handles the mechanism.

Source: Answer manual.

Exercise 54

Question

In the solution to the dining philosophers problem (Fig. 2-47), why is the state variable set to HUNGRY in the procedure take forks?

Answer

If a philosopher blocks, neighbors can later see that she is hungry by checking his state, in test, so he can be awakened when the forks are available.

Source: Answer manual.

Exercise 55

Question

Consider the procedure put forks in Fig. 2-47. Suppose that the variable state[i] was set to THINKING after the two calls to test, rather than before. How would this change affect the solution?

Answer

The change would mean that after a philosopher stopped eating, neither of his neighbors could be chosen next. In fact, they would never be chosen. Suppose that philosopher 2 finished eating. He would run test for philosophers 1 and 3, and neither would be started, even though both were hungry and both forks were available. Similarly, if philosopher 4 finished eating, philosopher 3 would not be started. Nothing would start him.

Source: Answer manual.

Exercise 56

Question

The readers and writers problem can be formulated in several ways with regard to which category of processes can be started when. Carefully describe three different variations of the problem, each one favoring (or not favoring) some category of processes. For each variation, specify what happens when a reader or a writer becomes ready to access the database, and what happens when a process is finished.

Answer

Variation 1: readers have priority. No writer may start when a reader is active. When a new reader appears, it may start immediately unless a writer is currently active. When a writer finishes, if readers are waiting, they are all started, regardless of the presence of waiting writers. Variation 2: Writers have priority. No reader may start when a writer is waiting. When the last active process finishes, a writer is started, if there is one; otherwise, all the readers (if any) are started. Variation 3: symmetric version. When a reader is active, new readers may start immediately. When a writer finishes, a new writer has priority, if one is waiting. In other words, once we have started reading, we keep reading until there are no readers left. Similarly, once we have started writing, all pending writers are allowed to run.

Source: Answer manual.

Exercise 57

Question

Write a shell script that produces a file of sequential numbers by reading the last number in the file, adding 1 to it, and then appending it to the file. Run one instance of the script in the background and one in the foreground, each accessing the same file. How long does it take before a race condition manifests itself? What is the critical region? Modify the script to prevent the race. (Hint: use `ln file file.lock` to lock the data file.)

Answer

A possible shell script might be `if [ ! -f numbers ]; then echo 0 > numbers; fi count = 0 while (test $count != 200 ) do count='expr $count + 1' n='tail -1 numbers' expr $n + 1 »numbers done` Run the script twice simultaneously, by starting it once in the background (using `&`) and again in the foreground. Then examine the file `numbers`. It will probably start out looking like an orderly list of numbers, but at some point it will lose its orderliness, due to the race condition created by running two copies of the script. The race can be avoided by having each copy of the script test for and set a lock on the file before entering the critical area, and unlocking it upon leaving the critical area. This can be done like this: `if ln numbers numbers.lock then n='tail -1 numbers' expr $n + 1 »numbers rm numbers.lock fi` This version will just skip a turn when the file is inaccessible. Variant solutions could put the process to sleep, do busy waiting, or count only loops in which the operation is successful.

Source: Answer manual.

Exercise 58

Question

Assume that you have an operating system that provides semaphores. Implement a message system. Write the procedures for sending and receiving messages.

Answer

A semaphore-based message system can be built as a bounded mailbox. The mailbox has a circular buffer, a mutex for mutual exclusion, one counting semaphore for empty slots, and one counting semaphore for full slots.

```
struct mailbox {
    message buf[N];
    int in = 0, out = 0;
    semaphore mutex = 1;
    semaphore empty = N;
    semaphore full = 0;
};

void send(mailbox *m, message x) {
    down(&m->empty); /* wait for a free slot */
    down(&m->mutex); /* enter critical section */
    m->buf[m->in] = x;
    m->in = (m->in + 1) % N;
    up(&m->mutex);
}
```

```

        up(&m->full); /* one more message available */
    }

    message receive(mailbox *m) {
        message x;
        down(&m->full); /* wait for a message */
        down(&m->mutex); /* enter critical section */
        x = m->buf[m->out];
        m->out = (m->out + 1) % N;
        up(&m->mutex);
        up(&m->empty); /* one more free slot */
        return x;
    }

```

Correctness: **empty** prevents overflow, **full** prevents underflow, and **mutex** protects the buffer indexes. This is a blocking message system: senders wait when the mailbox is full and receivers wait when it is empty.

Source: Independently prepared technical answer.

Exercise 59

Question

Solve the dining philosophers problem using monitors instead of semaphores.

Answer

A monitor solution keeps all fork-state changes inside the monitor and uses one condition variable per philosopher. A philosopher waits until neither neighbor is eating.

```

monitor DiningPhilosophers {
    enum { THINKING, HUNGRY, EATING } state[5];
    condition self[5];

    int left(int i) { return (i + 4) % 5; }
    int right(int i) { return (i + 1) % 5; }

    void test(int i) {
        if (state[i] == HUNGRY &&
            state[left(i)] != EATING &&
            state[right(i)] != EATING) {
            state[i] = EATING;

```

```

        signal(self[i]);
    }
}

void pickup(int i) {
    state[i] = HUNGRY;
    test(i);
    if (state[i] != EATING) wait(self[i]);
}

void putdown(int i) {
    state[i] = THINKING;
    test(left(i));
    test(right(i));
}
}

```

The monitor gives mutual exclusion automatically for `pickup`, `putdown`, and `test`. Deadlock is avoided because a philosopher eats only after both forks are available. Starvation depends on the monitor's scheduling policy; a fair condition-variable queue avoids indefinite postponement.

Source: Independently prepared technical answer.

Exercise 60

Question

Suppose that a university wants to show off how politically correct it is by applying the U.S. Supreme Court's "Separate but equal is inherently unequal" doctrine to gender as well as race, ending its long-standing practice of gender-segregated bathrooms on campus. However, as a concession to tradition, it decrees that when a woman is in a bathroom, other women may enter, but no men, and vice versa. A sign with a sliding marker on the door of each bathroom indicates which of three possible states it is currently in: Empty; Women present; Men present. In some programming language you like, write the following procedures: woman wants to enter, man wants to enter, woman leaves, man leaves. You may use whatever counters and synchronization techniques you like.

Answer

This is the same pattern as a readers-writers lock with two classes instead of readers and writers. A simple solution keeps the current bathroom state, the number currently inside, and a mutex plus condition variables.

```

enum State { EMPTY, WOMEN, MEN };
State state = EMPTY;
int inside = 0;
mutex m;
condition women_q, men_q;

void woman_wants_to_enter() {
    lock(m);
    while (state == MEN) wait(women_q, m);
    state = WOMEN;
    inside++;
    unlock(m);
}

void man_wants_to_enter() {
    lock(m);
    while (state == WOMEN) wait(men_q, m);
    state = MEN;
    inside++;
    unlock(m);
}

void woman_leaves() {
    lock(m);
    inside--;
    if (inside == 0) {
        state = EMPTY;
        broadcast(men_q);
        broadcast(women_q);
    }
    unlock(m);
}

void man_leaves() {
    lock(m);
    inside--;
    if (inside == 0) {
        state = EMPTY;
        broadcast(women_q);
        broadcast(men_q);
    }
    unlock(m);
}

```

Expected behavior: any number of people of the same gender may be inside together, but the other gender waits until the bathroom becomes empty. This version is safe but not strongly fair: continuous arrivals of one gender can delay the other. A fair version would keep waiting counts and alternate preference when the bathroom becomes empty.

Source: Independently prepared technical answer.

Exercise 61

Question

Rewrite the program of Fig. 2-23 to handle more than two processes.

Answer

The clean generalization for more than two processes is to use a known *filter lock*. It is a direct generalization of two-process mutual exclusion using levels and victims.

```
const int N = number_of_processes;
int level[N] = {0};
int victim[N]; /* victim[1] through victim[N-1] are used */

void lock(int me) {
    for (int L = 1; L < N; L++) {
        level[me] = L;
        victim[L] = me;
        while (exists k != me such that
                level[k] >= L && victim[L] == me) {
            ; /* busy wait */
        }
    }
}

void unlock(int me) {
    level[me] = 0;
}
```

At level L, at least one process is filtered out from advancing. Therefore at most one process reaches the critical section. This is a busy-waiting solution, so it is mainly useful for illustrating mutual exclusion, not for building a production kernel or application lock.

Source: Independently prepared technical answer.

Exercise 62

Question

Write a producer-consumer problem that uses threads and shares a common buffer. However, do not use semaphores or any other synchronization primitives to guard the shared data structures. Just let each thread access them when it wants to. Use sleep and wakeup to handle the full and empty conditions. See how long it takes for a fatal race condition to occur. For example, you might have the producer print a number once in a while. Do not print more than one number every minute because the I/O could affect the race conditions.

Answer

This experiment intentionally omits mutual exclusion, so it is expected to fail eventually. I did not execute it. A typical unsafe producer-consumer outline is:

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

#define N 100
int buffer[N];
int count = 0, in = 0, out = 0;

void *producer(void *arg) {
    int item = 0;
    for (;;) {
        while (count == N) usleep(1000);
        buffer[in] = item++;
        in = (in + 1) % N;
        count++; /* race */
    }
}

void *consumer(void *arg) {
    for (;;) {
        while (count == 0) usleep(1000);
        int item = buffer[out];
        out = (out + 1) % N;
        count--; /* race */
        (void)item;
    }
}
```

Expected behavior: because `count`, `in`, and `out` are shared without locks or semaphores, updates can be lost or observed inconsistently. The buffer can appear empty when it is not, full when it is not, or indexes can overtake each other. Safer testing: run only in a disposable test process, add a maximum iteration count, and print rare consistency checks rather than printing continuously. The correct fix is to guard shared state with a mutex and use condition variables or semaphores for full/empty conditions.

Source: Independently prepared technical answer.

Exercise 63

Question

A process can be put into a round-robin queue more than once to give it a higher priority. Running multiple instances of a program each working on a different part of a data pool can have the same effect. First write a program that tests a list of numbers for primality. Then devise a method to allow multiple instances of the program to run at once in such a way that no two instances of the program will work on the same number. Can you in fact get through the list faster by running multiple copies of the program? Note that your results will depend upon what else your computer is doing; on a personal computer running only instances of this program you would not expect an improvement, but on a system with other processes, you should be able to grab a bigger share of the CPU this way.

Answer

One safe way to let multiple instances work on the same list is to use a shared task file and claim work items atomically with a lock file. Each worker repeatedly takes one unclaimed number, tests it, and records the result.

```
#!/bin/sh
# worker.sh tasks.txt results.txt
tasks=$1
results=$2
lock="$tasks.lock"

is_prime() {
    n=$1
    [ "$n" -lt 2 ] && return 1
    d=2
    while [ $((d*d)) -le "$n" ]; do
        [ $((n%d)) -eq 0 ] && return 1
        d=$((d+1))
    done
```

```

        return 0
    }

    while ;; do
        while ! mkdir "$lock" 2>/dev/null; do sleep 0.01; done
        n=$(head -n 1 "$tasks")
        if [ -z "$n" ]; then rmdir "$lock"; exit 0; fi
        tail -n +2 "$tasks" > "$tasks.tmp" && mv "$tasks.tmp" "$tasks"
        rmdir "$lock"

        if is_prime "$n"; then echo "$n prime" >> "$results"; fi
    done

```

Run several workers on a disposable task file:

```

seq 2 100000 > tasks.txt
: > results.txt
sh worker.sh tasks.txt results.txt &
sh worker.sh tasks.txt results.txt &
wait

```

Expected behavior: no two workers should test the same number because task removal is protected by the lock directory. Multiple copies can finish faster on a loaded multiuser system by getting more CPU share, or on a multicore system if workers run in parallel. On a single-user, CPU-bound system already using all cores, extra copies may not help and may even add overhead.

Source: Independently prepared technical answer.

Exercise 64

Question

The objective of this exercise is to implement a multithreaded solution to find if a given number is a perfect number. N is a perfect number if the sum of all its factors, excluding itself, is N ; examples are 6 and 28. The input is an integer, N . The output is true if the number is a perfect number and false otherwise. The main program will read the numbers N and P from the command line. The main process will spawn a set of P threads. The numbers from 1 to N will be partitioned among these threads so that two threads do not work on the same number. For each number in this set, the thread will determine if the number is a factor of N . If it is, it adds the number to a shared buffer that stores factors of N . The parent process waits till all the threads complete. Use the appropriate synchronization primitive here. The parent will then determine if the input number is perfect, that is, if N is a sum of all its factors and then report accordingly. (Note: You can

make the computation faster by restricting the numbers searched from 1 to the square root of N .)

Answer

Partition the candidate factors among P threads. Each thread computes a local sum of factors and then adds it to a shared total under a mutex. Searching only up to `sqrt(N)` is faster if factor pairs are added together.

```
#include <math.h>
#include <pthread.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>

long N, limit, total = 1;
int P;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *worker(void *arg) {
    long id = (long)arg;
    long local = 0;
    for (long d = 2 + id; d <= limit; d += P) {
        if (N % d == 0) {
            long other = N / d;
            local += d;
            if (other != d) local += other;
        }
    }
    pthread_mutex_lock(&lock);
    total += local;
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main(int argc, char **argv) {
    if (argc != 3) return 1;
    N = atol(argv[1]);
    P = atoi(argv[2]);
    limit = (long)sqrt((double)N);
    if (N == 1) total = 0;
    pthread_t t[P];
```

```

        for (long i = 0; i < P; i++) pthread_create(&t[i], NULL, worker,
(void *)i);
        for (int i = 0; i < P; i++) pthread_join(t[i], NULL);
        printf("%s\n", total == N ? "true" : "false");
        return 0;
    }

```

Compile and test:

```

cc -O2 -pthread perfect.c -lm -o perfect
./perfect 6 4
./perfect 28 4
./perfect 12 4

```

Expected behavior: 6 and 28 print **true**; 12 prints **false**. Synchronization is needed only when adding each local result to the shared total.

Source: Independently prepared technical answer.

Exercise 65

Question

Implement a program to count the frequency of words in a text file. The text file is partitioned into N segments. Each segment is processed by a separate thread that outputs the intermediate frequency count for its segment. The main process waits until all the threads complete; then it computes the consolidated word-frequency data based on the individual threads' output.

Answer

Split the input file into N byte ranges, adjust each range to word boundaries, and have each thread build a private word-frequency map. After all threads finish, the main thread merges the maps. Private per-thread maps avoid locking on every word.

```

from collections import Counter
import re
import threading

def count_segment(text, start, end, out, index):
    while start > 0 and text[start-1].isalnum():
        start += 1
    while end < len(text) and text[end:end+1].isalnum():
        end += 1

```

```

words = re.findall(r"[A-Za-z0-9']+", text[start:end].lower())
out[index] = Counter(words)

def parallel_word_count(path, nthreads):
    text = open(path, encoding="utf-8", errors="ignore").read()
    step = max(1, len(text) // nthreads)
    partial = [Counter() for _ in range(nthreads)]
    threads = []
    for i in range(nthreads):
        start = i * step
        end = len(text) if i == nthreads - 1 else (i + 1) * step
        t = threading.Thread(target=count_segment,
                             args=(text, start, end, partial, i))
        threads.append(t)
        t.start()
    for t in threads:
        t.join()
    total = Counter()
    for c in partial:
        total.update(c)
    return total

```

Expected behavior: each segment produces an intermediate count, and the main thread consolidates them after joining. Safety and correctness notes: do not split in the middle of a word without boundary adjustment; for very large files, use memory mapping or chunked reading instead of loading the whole file; and do not update one shared dictionary from all threads without a lock.

Source: Independently prepared technical answer.